

Практическое задание 5. Рекомендательные системы — решение

Отдельный ноутбук-решение для practice-5-recommendations.ipynb. Код разбит по ячейкам в порядке заданий: загрузка данных, метрика, memory-based методы, user/item-based рекомендации, latent factor model и content-based модель.

Файлы данных в этой папке лежат рядом с ноутбуком и продублированы в download, поэтому путь к данным определяется автоматически.

0. Импорты и настройки

```
In [1]: from pathlib import Path
import json
import time

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_squared_error

RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

```
In [2]: def resolve_data_path():
    """Находит папку с данными независимо от того, где они лежат: рядом, в data или в download."""
    candidates = [Path('.'), Path('data'), Path('download')]
    required = ['catalogue.json', 'transactions.csv', 'ratings.csv', 'bookmarks.csv']
    for candidate in candidates:
        if all((candidate / name).exists() for name in required):
            return candidate
    raise FileNotFoundError('Не удалось найти файлы данных рядом с ноутбуком, в data/ или download/.')

DATA_PATH = resolve_data_path()
print('DATA_PATH =', DATA_PATH.resolve())
```

```
DATA_PATH = F:\University\Методы анализа данных\2 Семестр\Пр5
```

1. Загрузка данных

```
In [3]: catalogue = pd.read_json(DATA_PATH / 'catalogue.json').T
catalogue.index = catalogue.index.astype(np.uint16)
catalogue['element_uid'] = catalogue.index.astype(np.uint16)

catalogue.head()
```

```
In [4]: transactions = pd.read_csv(
    DATA_PATH / 'transactions.csv',
    dtype={
        'element_uid': np.uint16,
        'user_uid': np.uint32,
        'consumption_mode': 'category',
        'ts': np.float64,
        'watched_time': np.uint64,
        'device_type': np.uint8,
        'device_manufacturer': np.uint8,
    },
)

ratings = pd.read_csv(
    DATA_PATH / 'ratings.csv',
    dtype={
        'element_uid': np.uint16,
        'user_uid': np.uint32,
        'rating': np.uint8,
        'ts': np.float64,
    },
)

bookmarks = pd.read_csv(
    DATA_PATH / 'bookmarks.csv',
    dtype={
        'element_uid': np.uint16,
        'user_uid': np.uint32,
        'ts': np.float64,
    },
)

with open(DATA_PATH / 'test_users.json', 'r', encoding='utf-8') as f:
    final_test_users = json.load(f)['users']

print('catalogue:', catalogue.shape)
print('transactions:', transactions.shape)
print('ratings:', ratings.shape)
```

```
print('bookmarks:', bookmarks.shape)
print('final test users:', len(final_test_users))
```

```
catalogue: (10200, 10)
transactions: (9643012, 7)
ratings: (438790, 4)
bookmarks: (948216, 3)
final test users: 50000
```

2. Подготовка train/test

Целевая переменная — доля просмотренного времени. Длительность берется из catalogue, затем увеличивается на 10 минут и переводится в секунды, как в задании.

```
In [5]: duration_seconds = (catalogue['duration'].astype(float).fillna(0) + 10) * 60
transactions['duration'] = transactions['element_uid'].map(duration_seconds).astype(np.float32)
transactions['target'] = (transactions['watched_time'] / transactions['duration']).clip(0, 1).astype(np.float32)

transactions_small = transactions.loc[transactions.user_uid < transactions.user_uid.quantile(.01)].copy()
time_border = transactions_small.ts.quantile(.75)

transactions_train = transactions_small.loc[transactions_small.ts < time_border].reset_index(drop=True)
transactions_test = transactions_small.loc[transactions_small.ts > time_border].reset_index(drop=True)

print('transactions_small:', transactions_small.shape)
print('train:', transactions_train.shape, 'users:', transactions_train.user_uid.nunique(), 'items:', transactions_train.element_uid.nunique())
print('test:', transactions_test.shape, 'users:', transactions_test.user_uid.nunique(), 'items:', transactions_test.element_uid.nunique())

transactions_small: (96426, 9)
train: (72319, 9) users: 4498 items: 5124
test: (24107, 9) users: 2733 items: 3578
```

```
In [6]: train_pairs = transactions_train[['element_uid', 'user_uid']].drop_duplicates()
test_pairs = transactions_test[['element_uid', 'user_uid']].drop_duplicates()

ratings_train = ratings.merge(train_pairs.assign(in_train=True), on=['element_uid', 'user_uid'], how='left')
ratings_train = ratings_train.merge(test_pairs.assign(in_test=True), on=['element_uid', 'user_uid'], how='left')
ratings_train = ratings_train.loc[ratings_train['in_train'].fillna(False) & ~ratings_train['in_test'].fillna(False)]
ratings_train = ratings_train.drop(columns=['in_train', 'in_test']).reset_index(drop=True)

print('ratings after filtering:', ratings_train.shape)
```

```
ratings after filtering: (2836, 4)
```

Задание 0. MNAP@K

```
In [7]: def mnap_k(predictions, target, k=20):
        """Вычисляет MNAP@K для списков рекомендаций и множеств релевантных элементов."""
        if len(predictions) != len(target):
            raise ValueError('predictions and target must have the same length')

        user_scores = []
        for pred, true_items in zip(predictions, target):
            true_items = set(true_items)
            denom = min(len(true_items), k)
            if denom == 0:
                user_scores.append(0.0)
                continue

            hits = 0
            score = 0.0
            used = set()
            for rank, item in enumerate(list(pred)[:k], start=1):
                if item in used:
                    continue
                used.add(item)
                if item in true_items:
                    hits += 1
                    score += hits / rank
            user_scores.append(score / denom)

        return float(np.mean(user_scores)) if user_scores else 0.0
```

```
In [8]: test = [list(np.arange(1, 21)), list(np.arange(2, 22)), list(np.arange(3, 23))]
        target = [set(np.arange(1, 21)), set(np.arange(2, 22)), set(np.arange(3, 23))]
        assert mnap_k(test, target) == 1.0

        test = [list(np.arange(1, 21)), list(np.arange(2, 22)), list(np.arange(3, 23))]
        target = [set(np.arange(1, 11)), set(np.arange(2, 12)), set(np.arange(3, 13))]
        assert mnap_k(test, target) == 1.0

        test = [list(np.arange(1, 21)), list(np.arange(2, 22)), list(np.arange(3, 23))]
        target = [set(np.arange(2, 21)), set(np.arange(2, 22)), set(np.arange(3, 23))]
        assert round(mnap_k(test, target), 3) == 0.954
        print('Проверки MNAP пройдены')
```

Проверки MNAP пройдены

Задание 1. MemoryBased

В формуле Пирсона средние можно считать несколькими способами. Для полного расчета ниже используется быстрый режим `observed`: каждая строка центрируется по своим ненулевым значениям и нормируется один раз. Режим `pairwise_observed` повторяет первую контрольную трактовку из исходного ноутбука, а `common` — вторую трактовку со средними только по общему пересечению.

```
In [9]: class MemoryBased:
    def __init__(self, ratings, mean_mode='observed'):
        self.ratings = np.asarray(ratings, dtype=np.float32)
        if self.ratings.ndim != 2:
            raise ValueError('ratings must be a 2D user-item matrix')
        if mean_mode not in {'common', 'observed', 'pairwise_observed'}:
            raise ValueError("mean_mode must be 'common', 'observed' or 'pairwise_observed'")
        self.mean_mode = mean_mode
        self.binary_ratings = self.ratings > 0
        self.binary_ratings_float = self.binary_ratings.astype(np.float32)
        self.item_popularity = self.binary_ratings.sum(axis=0).astype(np.float32)
        self.popular_items = np.argsort(-self.item_popularity)
        self._item_similarity_cache = {}
        self._item_similarity_matrix = None

        self.centered_ratings, self.row_norms, self.normalized_ratings = self._center_rows_observed(self.ratings)

    @staticmethod
    def _center_rows_observed(matrix):
        """Центрирует каждую строку по среднему среди ненулевых наблюдений и нормирует ее."""
        matrix = np.asarray(matrix, dtype=np.float32)
        mask = matrix > 0
        counts = mask.sum(axis=1).astype(np.float32)
        sums = matrix.sum(axis=1)
        means = np.divide(sums, counts, out=np.zeros_like(sums, dtype=np.float32), where=counts > 0)
        centered = (matrix - means[:, None]) * mask
        norms = np.sqrt((centered ** 2).sum(axis=1)).astype(np.float32)
        normalized = np.divide(centered, norms[:, None], out=np.zeros_like(centered, dtype=np.float32), where=norms[:, None] > 0)
        return centered, norms, normalized

    @staticmethod
    def _observed_mean(matrix):
        mask = matrix > 0
        counts = mask.sum(axis=1)
        sums = matrix.sum(axis=1)
        return np.divide(sums, counts, out=np.zeros_like(sums, dtype=np.float32), where=counts > 0)
```

```

def _pearson_pairwise_observed_against_rows(self, matrix, vector):
    """Считает Пирсона на пересечении, но средние берет по всем ненулевым значениям строк."""
    matrix = np.asarray(matrix, dtype=np.float32)
    vector = np.asarray(vector, dtype=np.float32).reshape(-1)
    if vector.size != matrix.shape[1] or np.count_nonzero(vector) == 0:
        return np.zeros(matrix.shape[0], dtype=np.float32)

    common = (matrix > 0) & (vector > 0)
    counts = common.sum(axis=1)
    valid = counts >= 2
    result = np.zeros(matrix.shape[0], dtype=np.float32)
    if not np.any(valid):
        return result

    vector_mask = vector > 0
    x_mean = vector[vector_mask].mean() if np.any(vector_mask) else 0.0
    y_mean = self._observed_mean(matrix)
    x_centered = (vector[None, :] - x_mean) * common
    y_centered = (matrix - y_mean[:, None]) * common
    numerator = (x_centered * y_centered).sum(axis=1)
    denominator = np.sqrt((x_centered ** 2).sum(axis=1)) * np.sqrt((y_centered ** 2).sum(axis=1))
    result[valid] = np.divide(numerator[valid], denominator[valid], out=np.zeros_like(numerator[valid]), where=denominator[valid] > 0)
    return result

def _pearson_common_against_rows(self, matrix, vector):
    matrix = np.asarray(matrix, dtype=np.float32)
    vector = np.asarray(vector, dtype=np.float32).reshape(-1)
    if vector.size != matrix.shape[1] or np.count_nonzero(vector) == 0:
        return np.zeros(matrix.shape[0], dtype=np.float32)

    common = (matrix > 0) & (vector > 0)
    counts = common.sum(axis=1)
    valid = counts >= 2
    result = np.zeros(matrix.shape[0], dtype=np.float32)
    if not np.any(valid):
        return result

    x_sum = common @ vector
    y_sum = (matrix * common).sum(axis=1)
    x_mean = np.divide(x_sum, counts, out=np.zeros_like(x_sum, dtype=np.float32), where=counts > 0)
    y_mean = np.divide(y_sum, counts, out=np.zeros_like(y_sum, dtype=np.float32), where=counts > 0)
    x_centered = (vector[None, :] - x_mean[:, None]) * common
    y_centered = (matrix - y_mean[:, None]) * common
    numerator = (x_centered * y_centered).sum(axis=1)
    denominator = np.sqrt((x_centered ** 2).sum(axis=1)) * np.sqrt((y_centered ** 2).sum(axis=1))
    result[valid] = np.divide(numerator[valid], denominator[valid], out=np.zeros_like(numerator[valid]), where=denominator[valid] > 0)

```

```

        return result

def _pearson_observed_against_rows(self, normalized_matrix, vector):
    _, norms, normalized_vector = self._center_rows_observed(np.asarray(vector, dtype=np.float32).reshape(1, -1))
    if norms[0] == 0:
        return np.zeros(normalized_matrix.shape[0], dtype=np.float32)
    return (normalized_vector @ normalized_matrix.T).ravel().astype(np.float32)

def user_similarity(self, test_user):
    """Возвращает сходства одного пользователя со всеми пользователями обучающей матрицы."""
    if self.mean_mode == 'common':
        return self._pearson_common_against_rows(self.ratings, test_user)
    if self.mean_mode == 'pairwise_observed':
        return self._pearson_pairwise_observed_against_rows(self.ratings, test_user)
    return self._pearson_observed_against_rows(self.normalized_ratings, test_user)

def item_similarity(self, test_item):
    """Возвращает сходства одного фильма со всеми фильмами обучающей матрицы."""
    if np.isscalar(test_item):
        idx = int(test_item)
        if idx < 0 or idx >= self.ratings.shape[1]:
            return np.zeros(self.ratings.shape[1], dtype=np.float32)
        test_item = self.ratings[:, idx]
    if self.mean_mode == 'common':
        return self._pearson_common_against_rows(self.ratings.T, test_item)
    if self.mean_mode == 'pairwise_observed':
        return self._pearson_pairwise_observed_against_rows(self.ratings.T, test_item)
    normalized_items = self._get_normalized_items()
    return self._pearson_observed_against_rows(normalized_items, test_item)

def _get_normalized_items(self):
    if not hasattr(self, '_normalized_items'):
        _, _, self._normalized_items = self._center_rows_observed(self.ratings.T)
    return self._normalized_items

def _fallback_recommendations(self, user_vector, k):
    seen = np.asarray(user_vector).reshape(-1) > 0
    candidates = [idx for idx in self.popular_items if not seen[idx]]
    return np.array(candidates[:min(k, len(candidates))], dtype=int)

```

```

In [10]: I = np.array([
    [0.5, 0.4, 0, 0.1],
    [0, 0.1, 0.2, 0.5],
    [0.5, 0.5, 0.4, 0],
    [0.5, 0.4, 0.5, 0.1],
], dtype=np.float32)

```

```

pairwise_observed_model = MemoryBased(I, mean_mode='pairwise_observed')
result = pairwise_observed_model.user_similarity(np.array([0.5, 0.4, 0, 0.1], dtype=np.float32))
assert np.allclose(np.round(result, 2), np.array([1.0, -0.94, 0.92, 0.97]))

common_model = MemoryBased(I, mean_mode='common')
result = common_model.user_similarity(np.array([0.5, 0.4, 0, 0.1], dtype=np.float32))
assert np.allclose(np.round(result, 2), np.array([1.0, -1.0, 0.0, 1.0]))

fast_model = MemoryBased(I, mean_mode='observed')
result = fast_model.user_similarity(np.array([0.5, 0.4, 0, 0.1], dtype=np.float32))
assert np.isfinite(result).all()
print('Проверки MemoryBased пройдены')

```

Проверки MemoryBased пройдены

Задание 2. UserBased

```

In [11]: class UserBased(MemoryBased):
    def recommendation_k(self, user_vectors, k=20, batch_size=128):
        """Строит top-k рекомендаций для всех переданных пользователей без ограничения их числа."""
        user_vectors = np.asarray(user_vectors, dtype=np.float32)
        if user_vectors.ndim == 1:
            user_vectors = user_vectors.reshape(1, -1)

        recommendations = []
        for start in range(0, len(user_vectors), batch_size):
            batch = user_vectors[start:start + batch_size]
            _, norms, normalized_batch = self._center_rows_observed(batch)
            similarities = normalized_batch @ self.normalized_ratings.T
            similarities[norms == 0] = 0
            neighbours = (similarities > 0).astype(np.float32)
            neighbour_counts = neighbours.sum(axis=1)
            scores = neighbours @ self.binary_ratings_float
            scores = np.divide(scores, neighbour_counts[:, None], out=np.zeros_like(scores), where=neighbour_counts[:, None] > 0)

            for row_scores, user_vector, count in zip(scores, batch, neighbour_counts):
                if count == 0:
                    recs = self._fallback_recommendations(user_vector, k)
                else:
                    row_scores = row_scores.copy()
                    row_scores[user_vector > 0] = -np.inf
                    finite_count = np.isfinite(row_scores).sum()
                    recs = np.argsort(-row_scores)[:min(k, finite_count)].astype(int)
            recommendations.append(recs)

```

```
return np.array(recommendations, dtype=int) if recommendations and len({len(x) for x in recommendations}) == 1 else recommendations
```

```
In [12]: user_based = UserBased(I)
result = user_based.recomendation_k(np.array([0.5, 0.4, 0, 0.1], dtype=np.float32), k=1)
assert np.all(result == np.array([[2]]))
print('Проверки UserBased пройдены')
```

Проверки UserBased пройдены

Задание 3. ItemBased

```
In [13]: class ItemBased(MemoryBased):
    def _get_item_similarity_matrix(self):
        """Один раз считает полную матрицу сходств фильмов, после чего переиспользует ее."""
        if self._item_similarity_matrix is None:
            normalized_items = self._get_normalized_items()
            sim = normalized_items @ normalized_items.T
            sim = sim.astype(np.float32)
            sim[sim < 0] = 0
            np.fill_diagonal(sim, 1.0)
            self._item_similarity_matrix = sim
        return self._item_similarity_matrix

    def _item_similarity_by_index(self, item_idx):
        sim = self._get_item_similarity_matrix()
        item_idx = int(item_idx)
        if item_idx < 0 or item_idx >= sim.shape[0]:
            return np.zeros(sim.shape[0], dtype=np.float32)
        return sim[item_idx]

    def recomendation_k(self, user_vectors, k=20):
        """Строит top-k рекомендаций на основе максимального сходства с уже просмотренными фильмами."""
        user_vectors = np.asarray(user_vectors, dtype=np.float32)
        if user_vectors.ndim == 1:
            user_vectors = user_vectors.reshape(1, -1)

        sim = self._get_item_similarity_matrix()
        recommendations = []
        for user_vector in user_vectors:
            seen_items = np.flatnonzero(user_vector > 0)
            if len(seen_items) == 0:
                recommendations.append(self._fallback_recommendations(user_vector, k))
            continue
```

```

        scores = sim[seen_items].max(axis=0)
        scores = scores.copy()
        scores[seen_items] = -np.inf
        if not np.any(np.isfinite(scores)):
            recs = self._fallback_recommendations(user_vector, k)
        else:
            tie_break = self.item_popularity / (self.item_popularity.max() + 1e-8) * 1e-6
            recs = np.argsort(-(scores + tie_break))[:min(k, np.isfinite(scores).sum())]
            recommendations.append(recs.astype(int))

    return np.array(recommendations, dtype=int) if recommendations and len({len(x) for x in recommendations}) == 1 else recommendations

```

```

In [14]: item_based = ItemBased(I)
result = item_based.recommendation_k(np.array([0.5, 0.4, 0, 0.1], dtype=np.float32), k=1)
assert result[0, 0] == 2
print('Проверки ItemBased пройдены')

```

Проверки ItemBased пройдены

Задание 4. Матрицы R и оценка User-Based / Item-Based

```

In [15]: def build_user_item_matrix(frame, user_ids=None, item_ids=None):
        """Формирует плотную матрицу R: строки — пользователи, столбцы — фильмы."""
        if user_ids is None:
            user_ids = np.sort(frame['user_uid'].unique())
        if item_ids is None:
            item_ids = np.sort(frame['element_uid'].unique())

        pivot = frame.pivot_table(index='user_uid', columns='element_uid', values='target', aggfunc='max', fill_value=0)
        pivot = pivot.reindex(index=user_ids, columns=item_ids, fill_value=0).fillna(0).astype(np.float32)
        return pivot, np.asarray(user_ids), np.asarray(item_ids)

train_matrix_df, train_user_ids, item_ids = build_user_item_matrix(transactions_train)
test_user_ids = np.sort(transactions_test['user_uid'].unique())
test_user_vectors_df = train_matrix_df.reindex(index=test_user_ids, columns=item_ids, fill_value=0).fillna(0).astype(np.float32)

target_dict = transactions_test.groupby('user_uid')['element_uid'].agg(lambda s: set(map(int, s)))
target_sets = [target_dict.get(user, set()) for user in test_user_ids]

print('R_train:', train_matrix_df.shape)
print('test users:', len(test_user_ids))

```

R_train: (4498, 5124)
test users: 2733

```
In [16]: def indices_to_element_ids(predictions, item_ids):
        """Переводит индексы столбцов матрицы R обратно в element_uid."""
        item_ids = np.asarray(item_ids)
        return [[int(item_ids[idx]) for idx in recs] for recs in predictions]
```

```
In [17]: start = time.time()
user_model = UserBased(train_matrix_df.values, mean_mode='observed')
user_pred_idx = user_model.recommendation_k(test_user_vectors_df.values, k=20)
user_pred = indices_to_element_ids(user_pred_idx, item_ids)
user_mnap = mnap_k(user_pred, target_sets, k=20)
print(f'User-Based MNAP@20 on {len(target_sets)} users: {user_mnap:.5f} ({time.time() - start:.1f}s)')
```

User-Based MNAP@20 on 2733 users: 0.01904 (0.7s)

```
In [18]: start = time.time()
item_model = ItemBased(train_matrix_df.values, mean_mode='observed')
item_pred_idx = item_model.recommendation_k(test_user_vectors_df.values, k=20)
item_pred = indices_to_element_ids(item_pred_idx, item_ids)
item_mnap = mnap_k(item_pred, target_sets, k=20)
print(f'Item-Based MNAP@20 on {len(target_sets)} users: {item_mnap:.5f} ({time.time() - start:.1f}s)')
```

Item-Based MNAP@20 on 2733 users: 0.00608 (0.8s)

Задание 5. Latent Factor Model через ALS

Для фиксированной матрицы P шаг перенастройки элемента q_i получается симметрично пользовательскому шагу:

$$B_i = P[i]^T P[i], \quad c_i = P[i]^T r_i, \quad q_i = (\mu n_i I + B_i)^{-1} c_i,$$

где $P[i]$ — матрица факторов пользователей, взаимодействовавших с элементом i , а n_i — число таких пользователей.

```
In [19]: def latent_factor(train, target, test, lambda_, mu, N, K, P=None, Q=None, k=20, random_state=RANDOM_STATE):
        """Обучает LFM через ALS и возвращает рекомендации element_uid для каждого пользователя из test."""
        train = train[['user_uid', 'element_uid']].copy()
        target = np.asarray(target, dtype=np.float32)

        user_ids = np.sort(train['user_uid'].unique())
        item_ids_lfm = np.sort(train['element_uid'].unique())
        user_to_idx = {int(u): idx for idx, u in enumerate(user_ids)}
        item_to_idx = {int(i): idx for idx, i in enumerate(item_ids_lfm)}

        user_idx = train['user_uid'].map(user_to_idx).to_numpy()
        item_idx = train['element_uid'].map(item_to_idx).to_numpy()
```

```

local_rng = np.random.default_rng(random_state)
P_int = 0.1 * local_rng.random((len(user_ids), K), dtype=np.float32)
Q_int = 0.1 * local_rng.random((len(item_ids_lfm), K), dtype=np.float32)

if P is not None:
    P = np.asarray(P, dtype=np.float32)
    for raw_user, idx in user_to_idx.items():
        if 0 <= raw_user < P.shape[0]:
            P_int[idx] = P[raw_user, :K]
        elif 1 <= raw_user <= P.shape[0]:
            P_int[idx] = P[raw_user - 1, :K]
if Q is not None:
    Q = np.asarray(Q, dtype=np.float32)
    for raw_item, idx in item_to_idx.items():
        if 0 <= raw_item < Q.shape[0]:
            Q_int[idx] = Q[raw_item, :K]
        elif 1 <= raw_item <= Q.shape[0]:
            Q_int[idx] = Q[raw_item - 1, :K]

user_groups = [[] for _ in range(len(user_ids))]
item_groups = [[] for _ in range(len(item_ids_lfm))]
for u, i, r in zip(user_idx, item_idx, target):
    user_groups[u].append((i, r))
    item_groups[i].append((u, r))

eye = np.eye(K, dtype=np.float32)
for _ in range(N):
    for u, interactions in enumerate(user_groups):
        if not interactions:
            continue
        idxs = np.array([i for i, _ in interactions], dtype=int)
        r = np.array([r for _, r in interactions], dtype=np.float32)
        Q_u = Q_int[idxs]
        A = Q_u.T @ Q_u + lambda_ * len(interactions) * eye
        d = Q_u.T @ r
        P_int[u] = np.linalg.solve(A, d)

    for i, interactions in enumerate(item_groups):
        if not interactions:
            continue
        idxs = np.array([u for u, _ in interactions], dtype=int)
        r = np.array([r for _, r in interactions], dtype=np.float32)
        P_i = P_int[idxs]
        A = P_i.T @ P_i + mu * len(interactions) * eye
        d = P_i.T @ r
        Q_int[i] = np.linalg.solve(A, d)

```

```

popularity_order = np.argsort(-np.bincount(item_idx, minlength=len(item_ids_lfm)))
predictions = []
for raw_user in test:
    raw_user = int(raw_user)
    if raw_user in user_to_idx:
        scores = P_int[user_to_idx[raw_user]] @ Q_int.T
        order = np.argsort(-scores)
    else:
        order = popularity_order
    top = order[:min(k, len(order))]
    predictions.append([int(item_ids_lfm[i]) for i in top])
return np.array(predictions, dtype=int) if predictions and len({len(x) for x in predictions}) == 1 else predictions

```

```

In [20]: toy_train = pd.DataFrame({'user_uid': [1, 1, 2, 2], 'element_uid': [1, 2, 3, 4]})
toy_target = np.array([0.1, 0.8, 0.2, 0.3], dtype=np.float32)
toy_test = np.array([1, 2])
P0 = 0.1 * np.ones((toy_train.user_uid.max(), 10), dtype=np.float32)
Q0 = 0.1 * np.ones((toy_train.element_uid.max(), 10), dtype=np.float32)

# В исходном шаблоне проверка просит k=20 при четырех элементах; здесь проверяем top-1.
toy_pred = latent_factor(toy_train, toy_target, toy_test, lambda_=0.2, mu=0.001, N=20, K=10, P=P0, Q=Q0, k=1)
assert toy_pred.shape == (2, 1)
print('toy predictions:', toy_pred.tolist())

```

toy predictions: [[2], [2]]

```

In [21]: lfm_train = transactions_train[['user_uid', 'element_uid']]
lfm_target = transactions_train['target'].to_numpy(np.float32)
lfm_test_users = test_user_ids
lfm_target_sets = target_sets

lfm_results = []
for K in [5, 10, 20]:
    start = time.time()
    pred = latent_factor(lfm_train, lfm_target, lfm_test_users, lambda_=0.2, mu=0.001, N=8, K=K, k=20)
    score = mnap_k(pred.tolist(), lfm_target_sets, k=20)
    elapsed = time.time() - start
    lfm_results.append({'K': K, 'MNAP@20': score, 'seconds': elapsed})
    print(f'LFM K={K:2d}: MNAP@20={score:.5f}, time={elapsed:.1f}s')

pd.DataFrame(lfm_results)

```

```

LFM K= 5: MNAP@20=0.00515, time=0.8s
LFM K=10: MNAP@20=0.00566, time=0.9s
LFM K=20: MNAP@20=0.00628, time=1.0s

```

Задание 6. Content-Based

Собираем признаки пары пользователь-фильм: признаки контента из catalogue, статистики элемента, статистики пользователя и признаки истории взаимодействий. Получается больше десяти содержательных признаков.

```
In [22]: def make_item_features(catalogue, transactions_train, ratings_train, bookmarks):
    """Формирует признаки фильма из каталога и агрегатов пользовательских действий."""
    features = pd.DataFrame(index=catalogue.index.copy())
    features['duration'] = catalogue['duration'].astype(float).fillna(0)
    for col in ['feature_1', 'feature_2', 'feature_3', 'feature_4', 'feature_5']:
        features[col] = pd.to_numeric(catalogue[col], errors='coerce').fillna(0)

    type_dummies = pd.get_dummies(catalogue['type'], prefix='type')
    availability = catalogue['availability'].apply(lambda x: x if isinstance(x, list) else [])
    features['has_purchase'] = availability.apply(lambda x: int('purchase' in x))
    features['has_rent'] = availability.apply(lambda x: int('rent' in x))
    features['has_subscription'] = availability.apply(lambda x: int('subscription' in x))
    features['availability_count'] = availability.apply(len)
    features['attributes_count'] = catalogue['attributes'].apply(lambda x: len(x) if isinstance(x, list) else 0)
    features = features.join(type_dummies)

    item_stats = transactions_train.groupby('element_uid').agg(
        item_interactions=('target', 'size'),
        item_mean_target=('target', 'mean'),
        item_sum_watch=('watched_time', 'sum'),
        item_unique_users=('user_uid', 'nunique'),
    )
    rating_stats = ratings_train.groupby('element_uid').agg(
        item_rating_mean=('rating', 'mean'),
        item_rating_count=('rating', 'size'),
    )
    bookmark_stats = bookmarks.groupby('element_uid').size().rename('item_bookmark_count')

    features = features.join(item_stats, how='left').join(rating_stats, how='left').join(bookmark_stats, how='left')
    return features.fillna(0).astype(np.float32)

def make_user_features(transactions_train):
    """Формирует признаки пользователя по его истории в обучающей части."""
    mode_counts = pd.crosstab(transactions_train['user_uid'], transactions_train['consumption_mode'], normalize='index')
    mode_counts = mode_counts.rename(columns={col: f'user_mode_{col}' for col in mode_counts.columns})
    features = transactions_train.groupby('user_uid').agg(
        user_interactions=('target', 'size'),
        user_mean_target=('target', 'mean'),
```

```

        user_total_watch=('watched_time', 'sum'),
        user_unique_items=('element_uid', 'nunique'),
        user_mean_duration=('duration', 'mean'),
        user_last_ts=('ts', 'max'),
        user_device_types=('device_type', 'nunique'),
        user_manufacturers=('device_manufacturer', 'nunique'),
    )
    features = features.join(mode_counts, how='left')
    return features.fillna(0).astype(np.float32)

```

```

item_features = make_item_features(catalogue, transactions_train, ratings_train, bookmarks)
user_features = make_user_features(transactions_train)
print('item features:', item_features.shape)
print('user features:', user_features.shape)

```

```

item features: (10200, 21)
user features: (4498, 11)

```

```

In [23]: def build_pair_features(pairs, user_features, item_features):
    """Объединяет признаки пользователя и фильма для набора пар user_uid-element_uid."""
    pairs = pairs[['user_uid', 'element_uid']].copy()
    left = pairs.join(user_features, on='user_uid')
    full = left.join(item_features, on='element_uid', rsuffix='_item')
    return full.drop(columns=['user_uid', 'element_uid']).fillna(0).astype(np.float32)

content_train_pairs = transactions_train[['user_uid', 'element_uid']]
X_content = build_pair_features(content_train_pairs, user_features, item_features)
y_content = transactions_train['target'].to_numpy(np.float32)

split = int(len(X_content) * 0.8)
X_train_cb, X_val_cb = X_content.iloc[:split], X_content.iloc[split:]
y_train_cb, y_val_cb = y_content[:split], y_content[split:]

linear_model = LinearRegression()
linear_model.fit(X_train_cb, y_train_cb)
linear_val = np.clip(linear_model.predict(X_val_cb), 0, 1)
print('LinearRegression RMSE:', np.sqrt(mean_squared_error(y_val_cb, linear_val)))

boosting_model = HistGradientBoostingRegressor(max_iter=120, learning_rate=0.08, max_leaf_nodes=31, random_state=RANDOM_STATE)
boosting_model.fit(X_train_cb, y_train_cb)
boost_val = np.clip(boosting_model.predict(X_val_cb), 0, 1)
print('HistGradientBoosting RMSE:', np.sqrt(mean_squared_error(y_val_cb, boost_val)))

```

```

LinearRegression RMSE: 0.38105895852922006
HistGradientBoosting RMSE: 0.3188323345569993

```

```

In [24]: def content_recommend(model, eval_users, item_features, user_features, transactions_train, k=20):
    """Ранжирует все фильмы для каждого тестового пользователя и возвращает top-k."""
    item_rank = item_features.sort_values(['item_interactions', 'item_mean_target', 'item_rating_mean'], ascending=False).index.to_numpy(dtype=int)
    seen_by_user = transactions_train.groupby('user_uid')['element_uid'].agg(lambda s: set(map(int, s))).to_dict()

    predictions = []
    for user in eval_users:
        seen = seen_by_user.get(int(user), set())
        candidates = [int(item) for item in item_rank if int(item) not in seen]
        if not candidates:
            predictions.append([])
            continue
        pairs = pd.DataFrame({'user_uid': int(user), 'element_uid': candidates})
        X = build_pair_features(pairs, user_features, item_features)
        scores = np.clip(model.predict(X), 0, 1)
        top = np.argsort(-scores)[:min(k, len(candidates))]
        predictions.append([candidates[idx] for idx in top])
    return predictions

content_eval_users = test_user_ids
content_target_sets = target_sets

start = time.time()
linear_pred = content_recommend(linear_model, content_eval_users, item_features, user_features, transactions_train, k=20)
linear_mnap = mnap_k(linear_pred, content_target_sets, k=20)
print(f'LinearRegression content MNAP@20 on {len(content_eval_users)} users: {linear_mnap:.5f} ({time.time() - start:.1f}s)')

start = time.time()
boost_pred = content_recommend(boosting_model, content_eval_users, item_features, user_features, transactions_train, k=20)
boost_mnap = mnap_k(boost_pred, content_target_sets, k=20)
print(f'HistGradientBoosting content MNAP@20 on {len(content_eval_users)} users: {boost_mnap:.5f} ({time.time() - start:.1f}s)')

```

```

LinearRegression content MNAP@20 on 2733 users: 0.01627 (11.5s)
HistGradientBoosting content MNAP@20 on 2733 users: 0.00017 (32.2s)

```